MISSION DE LA SÉANCE

# Faire parler le métier. Puis ouvrir une API.

Un domaine pur, des invariants testés et le même use case accessible depuis deux interfaces.

Architecture hexagonale · pratique ciblée

→ pour commencer



Photo Annie Spratt / Unsplash

LA CIBLE

# Votre cœur métier ne connaît ni Streamlit, ni FastAPI, ni base de données.

DOMAINE

### Concepts précis

Entités, Value Objects et vocabulaire de vos utilisateurs.

USE CASE

### Une action utile

Une orchestration stable indépendante de l’interface.

API

### Traduction HTTP

DTO, validation et codes d’erreur à la frontière.

SE REPÉRER AVANT DE CODER

# Se mettre d’accord sur les mots avant les routes.



Une règle métier doit être compréhensible par l’équipe.

À ÉCRIRE

## Le glossaire de votre domaine.

Partir d’une action utilisateur et nommer les objets, les valeurs et les refus possibles.

À CRÉER

## Un cœur sans framework.

Écrire les types dans `src/filrouge/domain/` et les règles dans leurs tests.

À RACCORDER

## Une action, deux interfaces.

Streamlit et FastAPI utilisent le même use case ; les DTO restent à la frontière HTTP.

Photo Annie Spratt / Unsplash · Pictogrammes : tracés Lucide, licence ISC.

AVANT LES CLASSES

# Écrivez un glossaire de 8 à 12 termes.

1

## Nommer les objets

Ce que les utilisateurs créent, suivent, classent ou évaluent.

2

## Nommer les règles

Ce qui rend un objet valide ou une transition autorisée.

3

## Nommer les actions

Les verbes utiles qui deviendront des use cases.

### Exemple enseignant

Musicothèque : morceau, artiste, album, playlist, note, ambiance, durée, écoute.

Votre groupe conserve ses propres mots et ses propres règles.

```
uv sync --extra rag --extra agent \
  --extra api --group dev
```

ENTITY · VALUE OBJECT

# Choisissez peu d’objets, mais donnez-leur de vraies responsabilités.

2–4

### Entités

Une identité stable et un cycle de vie : projet, morceau, commande…

1–3

### Value Objects

Une valeur définie par son contenu : note, période, montant…

2+

### Invariants

Des règles vraies dès la construction de l’objet.

- ✓ Objets immuables quand c’est pertinent.
- ✓ Erreurs métier compréhensibles.
- ✓ Aucun objet SDK dans le domaine.
- ✓ Tests directs, sans framework.

ATELIER · DOMAINE PUR

# Une valeur métier commence par deux tests.

01 **Prendre une règle du glossaire**

Exemple de secours : une note entière entre 1 et 5. Votre groupe choisit une règle de son domaine.

02 **Écrire le cas valide et le refus**

Créer un test pour chaque borne et une valeur hors borne. Les tests importent directement le domaine.

03 **Implémenter seulement la règle**

Créer un Value Object immuable ; valider à la construction. Ajouter ensuite les entités qui l'utilisent.

À CRÉER · tests/test_domain_rating.py · exemple partiel

```
import pytest
from filrouge.domain.music import Rating
from filrouge.domain.exceptions import ValidationError

def test_rating_accepts_boundary():
    assert Rating(1).value == 1

def test_rating_rejects_invalid_value():
    with pytest.raises(ValidationError):
        Rating(0)
```

**Vérifier.** Les tests fonctionnent sans serveur ni base. Adapter le nom de module et l'exception publique à votre domaine.

LES INVARIANTS D’ABORD

# Un test montre ce que le domaine accepte et refuse.

## Cas valides

- construction minimale correcte ;
- valeurs limites autorisées ;
- égalité d’un Value Object ;
- transition métier possible.

## Cas invalides

- identifiant ou libellé vide ;
- valeur hors borne ;
- doublon interdit ;
- transition incohérente.

```
uv run pytest -k domain
```

Le domaine se teste sans démarrer l’application

UNE ACTION MÉTIER STABLE

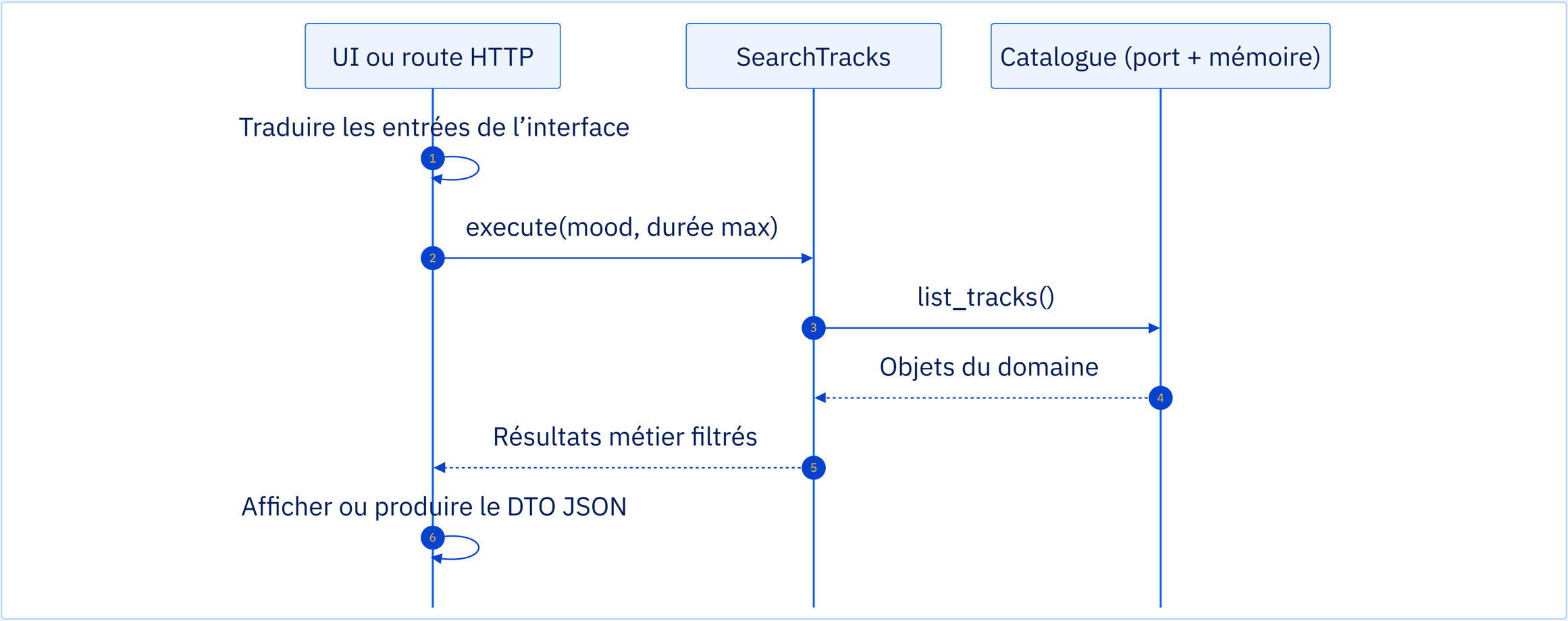
# Orchestrez via un port. Gardez la technologie dehors.




- ✓ Valider les entrées métier.
- ✓ Retourner des types du domaine.
- ✓ Tester avec un adaptateur en mémoire.
- ✓ Ne pas importer FastAPI.

FLUX À IMPLÉMENTER · SÉQUENCE

# L’UI et l’API délèguent à la même action.




L’adaptateur mémoire est injecté au bootstrap. Le use case reçoit des types simples et retourne des objets métier. Flèche pleine : appel ; pointillés : retour.

ATELIER · FRONTIÈRE HTTP

# Branchez la route après le use case testé.

01 **Créer le port et un fake mémoire**

Dans `application/ports/`, décrire la capacité. Tester le use case avec trois objets connus.

02 **Créer votre entrée API**

Dans `entrypoints/api.py`, définir le DTO et une factory `create_api(use_case)`.

03 **Traduire aux deux extrémités**

Lire les paramètres, appeler execute une fois, convertir les résultats en DTO. Le bootstrap compose l'application.

À CRÉER · entrypoints/api.py · squelette incomplet

```
from fastapi import FastAPI

def create_api(search_tracks):
    app = FastAPI()

    @app.get("/tracks/search")
    def search(mood: str):
        tracks = search_tracks.execute(mood)
        # À faire : DTO et erreurs HTTP.
        return [to_response(t) for t in tracks]

    return app
```

**Vérifier.** Le test injecte un use case et appelle la factory. Streamlit réutilise le même cas d'usage, sans appel HTTP imposé.

Étapes détaillées et commandes copiables dans le README du TP.

CAPSULE C6 · FRONTIÈRE HTTP

# La route traduit. Elle ne réécrit pas le métier.




400/422

## Validation

Une entrée incorrecte devient une réponse HTTP explicite.

200

## Succès

Le résultat métier est traduit en JSON stable.

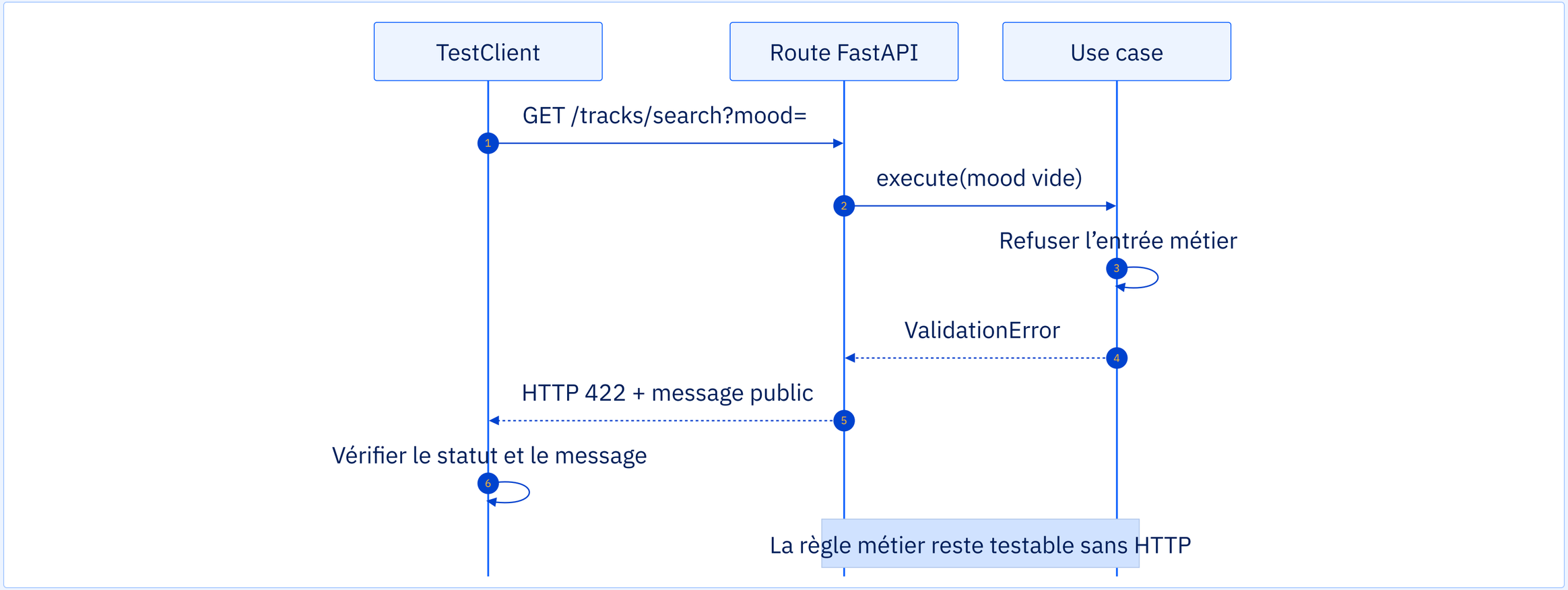
0

## Duplication

L’UI et l’API appellent le même objet use case.

FLUX À IMPLÉMENTER · SÉQUENCE

# Une erreur métier devient une réponse explicite.




Ce scénario choisit HTTP 422 pour une entrée métier invalide. La route traduit l’exception ; le test vérifie le contrat. Flèche pleine : appel ; pointillés : retour.

PARCOURS DE VALIDATION

# Une règle métier, deux interfaces, zéro logique copiée.

01

### Domaine

Montrer un objet valide puis un invariant refusé.

02

### UI + API

Exécuter le même use case avec les mêmes paramètres.

03

### Erreur HTTP

Envoyer une entrée invalide et expliquer le code reçu.

```
uv run pytest  &&  uv run ruff check .
git add .  &&  git commit -m "Construit le domaine et expose le use case par API"
```

AVANT DE QUITTER S4

# Le domaine est le centre. Les interfaces sont remplaçables.

## Vous avez construit

- un glossaire métier ;
- des entités et Value Objects ;
- des invariants testés ;
- un use case accessible par API.

## À conserver pour S5

- les identifiants métier stables ;
- les relations entre concepts ;
- les sources de vérité ;
- les tests du domaine.

```
uv run pytest  &&  uv run ruff check .
```

S4 · CONSIGNES À GARDER PROJETÉES

# À vous : un domaine, deux interfaces.

01 **Nommer le métier**

Écrire 8 à 12 termes et deux règles avec exemples valides et invalides.

02 **Modéliser et tester**

2 à 4 entités, 1 à 3 Value Objects ; tests directs des invariants, sans framework.

03 **Construire une action utile**

Port de capacité + fake mémoire + use case dans application/ ; tester avant l'API.

04 **Exposer et comparer**

Route et DTO dans entrypoints/ ; mêmes paramètres en UI/API, erreur HTTP testée.

## À montrer avant de terminer

- Un invariant accepté puis refusé.
- Le même use case appelé par UI et API.
- Une requête invalide et son statut HTTP documenté.

```
uv run pytest -k domain
uv run pytest
uv run ruff check .
```

Bloqué ? README de ce TP → « Pas à pas ». Partir d'une seule action et d'un adaptateur mémoire ; aucun graphe à construire ici.